

Un moteur autonome de type Thalès-Pandrosion pour l'extraction géométrique de racines polynomiales

Étude mathématique et expérimentale du script 113

Ivan Besevic

29 avril 2026

Abstract

Nous décrivons le moteur `113_pandrosion_autonomous_heuristic_thales_engine.py`, un extracteur autonome de racines complexes de systèmes polynomiaux denses. Le moteur ne suit pas tous les chemins de Bézout. Il construit plutôt un flot géométrique unique combinant homothétie de type Thalès, carte de Riemann-Möbius, optimisation du point de départ, puis correction locale dans les coordonnées transformées. Le point central est de déplacer l'effort numérique de l'énumération homotopique vers la géométrie d'initialisation. Sur les systèmes de Kostlan testés, le moteur retrouve toutes les racines demandées pour $ks(2, 12)$ et $ks(2, 14)$, c'est-à-dire respectivement 144/144 et 196/196 racines numériques distinctes sous le seuil de résidu utilisé. Le document formalise le flot, précise la nature non derivative-free du correcteur local de 113, et donne les résultats reproductibles observés.

1 Problème

Soit

$$F = (F_1, \dots, F_n) : \mathbb{C}^n \rightarrow \mathbb{C}^n$$

un système polynomial. Pour un système dense générique où chaque équation a degré total d , le nombre de Bézout est d^n . Une stratégie homotopique complète associe typiquement un chemin à chaque solution attendue. Pour des cas comme $ks(6, 16)$, cela donne

$$16^6 = 16\,777\,216$$

chemins théoriques. Le moteur 113 adopte un autre point de vue: extraire des racines utiles et distinctes par exploration géométrique directe, sans énumération exhaustive des chemins.

2 Systèmes de Kostlan utilisés

Les tests emploient une famille dense de type Kostlan. Pour n variables et degré d , on considère des monômes

$$z^\alpha = z_1^{\alpha_1} \cdots z_n^{\alpha_n}, \quad |\alpha| \leq d,$$

et des coefficients complexes aléatoires pondérés par des facteurs multinomiaux. Dans les sorties du script, le nombre de termes par polynôme est

$$\binom{n+d}{n}.$$

Par exemple, $ks(2, 14)$ contient 120 termes par polynôme, donc 240 termes au total pour deux équations.

3 Flot géométrique unique

Le moteur 113 génère un point initial non pas directement dans l'espace original z , mais dans une coordonnée auxiliaire u . Pour chaque coordonnée, il applique une transformation de Möbius couplée à une homothétie:

$$y_j = \Lambda p_j \frac{\cos(\theta_j)(u_j/p_j) + \sin(\theta_j)}{-\sin(\theta_j)(u_j/p_j) + \cos(\theta_j)}. \quad (1)$$

Ensuite,

$$z = Ay, \quad (2)$$

où A est une transformation linéaire diagonale ou quasi diagonale construite dans le moteur autonome.

Cette formule contient plusieurs régimes sans créer de branche algorithmique séparée:

$$\begin{aligned} \theta_j = 0 & \Rightarrow \text{carte affine homothétique,} \\ \theta_j \approx \pi/2 & \Rightarrow \text{régime d'inversion / carte vers l'infini,} \\ \Lambda \gg 1 & \Rightarrow \text{homothétie forte de type Thalès.} \end{aligned}$$

Le but n'est pas seulement de normaliser les variables, mais de produire des points situés dans des bassins numériques variés.

4 Optimisation du point de départ

Le moteur applique une étape *startopt*. On pose

$$G(y) = F(Ay).$$

À partir du point brut y_0 fourni par (1), on teste un petit ensemble de perturbations radiales et directionnelles, puis on garde le candidat qui minimise une énergie de démarrage de la forme

$$E(y) = \log(1 + \|G(y)\|) + \varepsilon \log(1 + \|y\|), \quad (3)$$

avec ε très petit. Géométriquement, cette étape correspond à choisir un meilleur point de départ afin de réduire le nombre de constructions proportionnelles successives nécessaires à la correction.

5 Correcteur local de 113

Il est important de caractériser correctement la nature du moteur 113. Après le flot géométrique, le correcteur local résout

$$J_G(y)\Delta = -G(y), \quad y \leftarrow y + \lambda\Delta, \quad (4)$$

où

$$J_G(y) = J_F(Ay)A. \quad (5)$$

Le scalaire λ est choisi par une recherche proportionnelle amortie.

Remarque 1. *Le moteur 113 n'est donc pas derivative-free. Sa géométrie globale est Pandrosion/Thalès, mais son correcteur local utilise un Jacobien. Le moteur ne fait toutefois pas appel à un solveur de racines caché: il n'utilise ni `numpy.roots`, ni `scipy`, ni `sympy`, ni homotopie externe. Le solveur linéaire `np.linalg.solve` sert uniquement à résoudre (4).*

6 Mécanisme heuristique

Le moteur 113 reste volontairement simple. À chaque essai, il choisit des paramètres

$$(\Lambda, \theta, p, r)$$

à partir d'une grille déterministe et d'une pression heuristique liée aux duplications et aux échecs. Lorsque des duplications apparaissent, la puissance homothétique est poussée vers d'autres échelles. Cette règle peut être vue comme une version discrète d'une exploration de bassins d'attraction:

$$\text{doublons fréquents} \Rightarrow \text{changer d'échelle ou de carte.}$$

La stratégie est unique: il ne s'agit pas d'une collection de politiques concurrentes.

7 Algorithme

Entrée: un système $F : \mathbb{C}^n \rightarrow \mathbb{C}^n$, un nombre cible N , un nombre maximal d'essais P .

Sortie: un ensemble R de racines numériques distinctes.

1. Construire le système dense et l'évaluateur F, J_F .
2. Initialiser $R = \emptyset$.
3. Pour $t = 0, \dots, P - 1$:
 - (a) générer une direction u_t , une homothétie Λ_t , un angle θ_t et un pôle p_t ;
 - (b) appliquer le flot (1)-(2);
 - (c) optimiser le point de départ par (3);
 - (d) corriger par (4);
 - (e) valider le résidu $\|F(z)\|$ et ajouter la racine si elle est distincte.
4. Arrêter lorsque $|R| = N$ ou lorsque le budget d'essais est épuisé.

8 Résultats expérimentaux

Les tableaux suivants reprennent les sorties JSON générées par le moteur 113. Les temps sont ceux de l'environnement interactif local et ne constituent pas un benchmark matériel normalisé.

Cas	Bézout	Racines demandées	Racines uniques	Essais	Doublons	Temps (s)
$ks(2, 12)$	144	144	144	393	132	2.422
$ks(2, 14)$	196	196	196	455	107	2.988
$ks(6, 16)$	16,777,216	1	1	1	0	0.797

Table 1: Résultats principaux du moteur 113.

Le moteur a également été essayé sur des cas multivariés complets ou partiels. Les observations rapportées sont:

- $ks(3, 4)$: 64/64 racines,
- $ks(4, 3)$: 81/81 racines,
- $ks(3, 6)$: 80/80 racines demandées,
- $ks(4, 4)$: 80/80 racines demandées,
- $ks(5, 3)$: 50/50 racines demandées,
- $ks(6, 4)$: 10/10 racines demandées.

Ces tests confirment que le code n'est pas spécialisé au cas bivarié.

Cas	Termes par polynôme	Termes totaux	Résidu max.	Résidu moyen
$ks(2, 12)$	91	182	$9.91 \cdot 10^{-9}$	$1.81 \cdot 10^{-9}$
$ks(2, 14)$	120	240	$9.96 \cdot 10^{-9}$	$1.79 \cdot 10^{-9}$
$ks(6, 16)$	74,613	447,678	$1.40 \cdot 10^{-11}$	$1.40 \cdot 10^{-11}$

Table 2: Résidus numériques des racines acceptées dans les tests 113.

9 Discussion

L'intérêt principal de 113 est de montrer qu'une forte géométrisation de l'initialisation peut remplacer, pour l'extraction de nombreuses racines, le suivi systématique des chemins de Bézout. La réussite sur $ks(2, 12)$ et $ks(2, 14)$ est particulièrement notable car l'algorithme ne construit pas explicitement les d^n chemins de départ. En revanche, 113 reste localement un moteur avec dérivées. Pour obtenir une version plus fidèle à l'idée de Pandrosion pur, il faut remplacer J_G par une matrice de pente exacte $Q(a, b)$. C'est précisément l'objet du moteur 115.

10 Reproductibilité

Commandes utilisées pour les cas principaux:

```
python -S 113_pandrosion_autonomous_heuristic_thales_engine.py \
--cases 2,12 --equation-normalize --count 144 --pool 5000 \
--epochs 24 --accept 1e-8 --cluster-sep 1e-8 \
--startopt-candidates 12
```

```
python -S 113_pandrosion_autonomous_heuristic_thales_engine.py \
--cases 2,14 --equation-normalize --count 196 --pool 7000 \
--epochs 24 --accept 1e-8 --cluster-sep 1e-8 \
--startopt-candidates 12
```

11 Conclusion

Le moteur 113 propose un schéma autonome, multivarié et sans dépendance à d'anciens scripts. Sa contribution est le flot géométrique unique combinant homothétie forte, cartes de Riemann-Möbius et optimisation du point de départ. Cette couche globale rend la correction locale très efficace sur les benchmarks de Kostlan testés. Sa limite mathématique est claire: le correcteur local est de type Jacobien. Cette limite motive naturellement une version Pandrosion pure fondée sur une matrice de pente exacte.